Assignment - 2

NLP: CS60075

Autumn Semester 2025, IIT Kharagpur

Building 'mini-Copilot': Sequential Code Generation with Recurrent Architectures

Point of contact - Paramananda Bhaskar (paramanandabhaskar@gmail.com)

Assignment Beginning: 11:00 PM, 8th September 2025
Assignment Deadline: 11:59 PM, 13th October 2025

Plagiarism Alert

- Plagiarism in any form is strictly prohibited.
- Both the SOURCE and the TARGET individuals will suffer a penalty of 100% if a plagiarism detector tool detects 60% or more code similarity.
- You are not supposed to share your assignment with anyone even after the deadline for submission is over.
- We shall use multiple AI detectors like GPT (and 3 more undisclosed), and if at least 3 of them flag your code as AI-generated then we shall deem the submission to be 100% plagiarism.

[**For this assignment, it is strongly recommended that you utilize free GPU Accelerators from Kaggle or Google Colab.** These will be highly beneficial for training your models within a reasonable timeframe.]

Overview

In this assignment, you will design, train and evaluate a token-level code completion system - your own *mini-Copilot*.

The primary objective is to develop a hands-on understanding of RNN and LSTM architectures. You will conduct a comparative analysis of their performance and then augment the best-performing model with an architectural improvement, which is crucial for building robust NLP systems.

Tasks

Task 1: Data Preparation **[10% of TOTAL]**

You will use the **Python subset** of the **CodeXGLUE Code Completion (Token-level)** dataset. The first step is to prepare this data for your models.

- **Preprocessing:** Create a vocabulary **from the train split** using the top **10,000** most frequent tokens, mapping all others to an out-of-vocabulary (OOV) token, e.g., <UNK>.
- **Sequencing:** Pad or truncate all code sequences to a fixed length of **1,000** tokens.
- **Create Validation Split**: Randomly split the training data into a new training set (**80%**) and a validation set (**20%**). Use a fixed **random seed** of **42** to ensure reproducibility.
- **Build Loaders:** Build efficient PyTorch DataLoaders for the training, validation and test splits.

Task 2: Baseline Models - Comparison **[55% of TOTAL]**

This is the core of the assignment, where you will implement, train and analyze two baseline models for token-level Python code completion.

2.1 Model Implementation

Implement two separate PyTorch models, each using a different recurrent architecture:

Required Architectures:

1. **RNN**: Using *torch.nn.RNN*
2. **LSTM**: Using *torch.nn.LSTM*

Shared Architectural Design:

Both models must conform to the following specifications:

- **Embedding Layer:**
  - Dimension = 256
  - You must choose **one** of two strategies:
    (i) Trainable from scratch: Initialize with random weights.
    (ii) Pre-trained embeddings: Initialize with embeddings from models like Word2Vec, GloVe or code-specific embeddings like code2vec.
  - In your report, you must mention and justify your choice.
- **Recurrent Stack:**
  - Number of layers: 2
  - Hidden units per layer: 512
  - Use the respective cell type for each model (RNN or LSTM).
  - Choose an activation function (e.g., `tanh`, `ReLU`) for the recurrent layers and justify your choice in the report.
- **Regularization:**
  - Apply a Dropout layer between recurrent layers and before the output layer. Choose a dropout probability (e.g., 0.2–0.5) and mention your choice in the report.

- **Output Layer:**
  - A fully connected (Linear) layer that maps the final hidden state to a vector of logits.
  - The output size must match the vocabulary size (10,000 most frequent tokens + any special tokens like <PAD> and <UNK>).

2.2 Training & Validation

You will train each model using a standardized procedure to ensure a fair comparison using:

- **Optimizer**: Adam
- **Loss Function**: nn.CrossEntropyLoss. **Remember to ignore the loss computed on <PAD> tokens (use the ignore_index parameter)**.
- **Training Duration**: Train the model either for a fixed number of epochs (decide based on dataset size and convergence speed) or until clear convergence/divergence is observed on the validation set (e.g., early stopping when validation loss stops improving).
- **Validation**: After each training epoch, you must evaluate the model on the held-out validation split of the dataset to monitor for overfitting and to select the best model checkpoint.

What to Measure on the Validation Set:

After each epoch, calculate the following metrics for each model:

Top-5 Accuracy:

- It measures how often the correct next token appears anywhere in the model's list of its top 5 best guesses. This is practical because code completion tools show developers a short list of suggestions.
- **Formula**: Top-5 Accuracy = (Number of correct predictions in the top 5) / (Total predictions) * 100%
- **Example**: If the correct next token is "numpy" and the model's top 5 predictions are ["os", "sys", "numpy", "math", "json"], this counts as a success. A high Top-5 Accuracy (e.g., 90%) means the correct suggestion is almost always in the list presented to a developer.

You should plot the validation Top-5 Accuracy for each model after every epoch to see how they learn and compare.

Perplexity (PPL):

- It measures how well a probability model predicts a sample and quantifies the model's uncertainty.
- **Formula:** Perplexity is the exponential of the average cross-entropy loss. *PPL=exp(CrossEntropyLoss)*
- **Interpretation:** A **lower** perplexity score indicates a better model, signifying less uncertainty in its predictions. For example, a perplexity of 10 means the model is as uncertain as if it were choosing from 10 equally likely options at each step.

2.3 Final Evaluation & Reporting

Once training is complete, you will perform one decisive test to see how well your models perform on completely new, unseen code.

Step-by-Step Instructions:

1. **Choose Your Best Model checkpoint:**
    - For each of your two models (RNN and LSTM), look back at your validation results.
    - Select the specific saved model from the epoch that achieved the highest validation Top-5 Accuracy. This is your best checkpoint for that architecture.
2. **Run the Final Test:**
    - Load each of these two best checkpoints.
    - Evaluate each one a single time on the held-out **test split**.
3. **Calculate the Final Metrics:**
    - For each model, calculate its final Test Top-5 Accuracy using the entire test set. Also, compute the perplexity (PPL) on the test split.
4. **Report Your Results:**
    - Present your findings in a clear table in the report.
    - In addition to Top-5 Accuracy and PPL, **for each of your two models (RNN** and **LSTM), report the total number of trainable parameters.**
    - Discuss whether a higher parameter count correlates with better performance in your experiments.
    - *(Optional)* You may also report training time efficiency of each of the models for additional insights.

Task 3: Residual Recurrent Models **[30% of TOTAL]**

In this part, you will enhance your best-performing model from Task 2 by integrating a **residual (skip) connection**. This architectural improvement helps mitigate the vanishing gradient problem and improves information flow, which is crucial for training deep networks effectively. You will implement this while keeping the model's depth consistent for a fair comparison.

3.1 Implementation

1. **Base Architecture:** Choose your **best-performing model from Task 2** (either RNN or LSTM) based upon the Test Top-5 Accuracy as in Task-2. The model will retain its 2-layer recurrent structure.
2. **Residual Connection:** Implement a residual connection that adds the output of the first recurrent layer to the output of the second recurrent layer.
    - This creates a shortcut for information, allowing the network to learn an identity function if a layer isn't helpful and enabling gradients to flow more easily.

- **Implementation**:
  - Pass the input sequence through your first recurrent layer to get its output (H1).
  - Pass the output of the first layer (H1) to the second recurrent layer, which will produce its own output (H2).
  - Your final recurrent output will be the **element-wise sum** of the first layer's output and the second layer's output: H_final=H1+H2.
  - The final linear layer for vocabulary prediction will then take this H_final as input.

3.2 Training

Train this new, residually-connected model **using the exact same protocol as in Task 2.**

- **Optimizer:** Adam
- **Loss Function:** nn.CrossEntropyLoss (remember to use the ignore_index parameter).
- **Training Duration:** Use the same stopping criteria or number of epochs as in Task 2.

3.3 Evaluation and Reporting

1. **Final Test:** Evaluate the trained, residually-connected model on the held-out test split.
2. **Comparison:** Compare its final Test Top-5 Accuracy and Perplexity (PPL) directly against your best baseline model from Task 2.
3. **Report:** In your report, mention the impact of adding the residual connection. Did it improve performance?
   - Alongside accuracy and PPL, report the **total number of trainable parameters** of the residual recurrent model.
   - *(Optional)* You may also report **training time efficiency**.

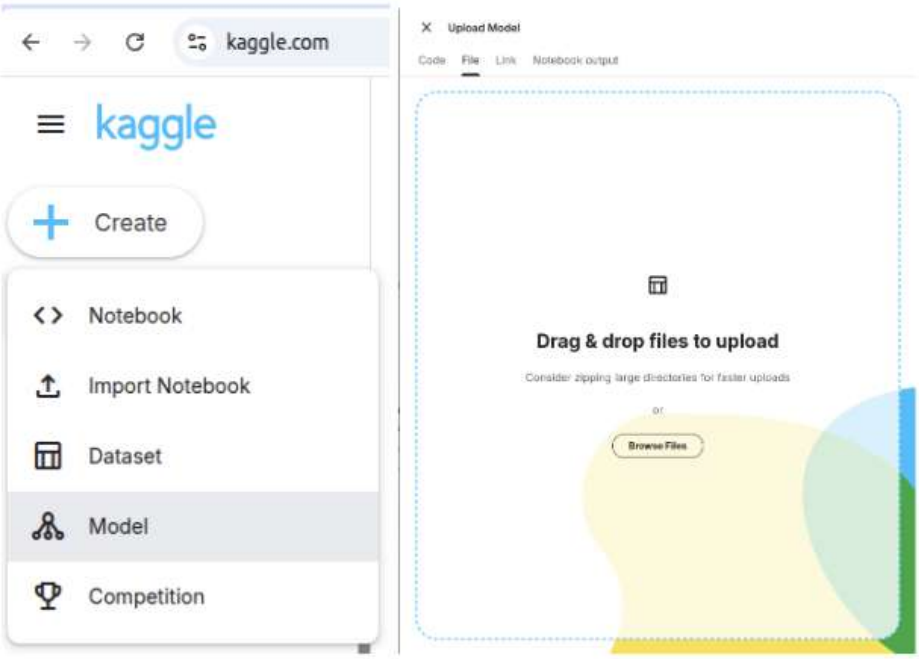
Task 4: Final Report **[5% of TOTAL]**

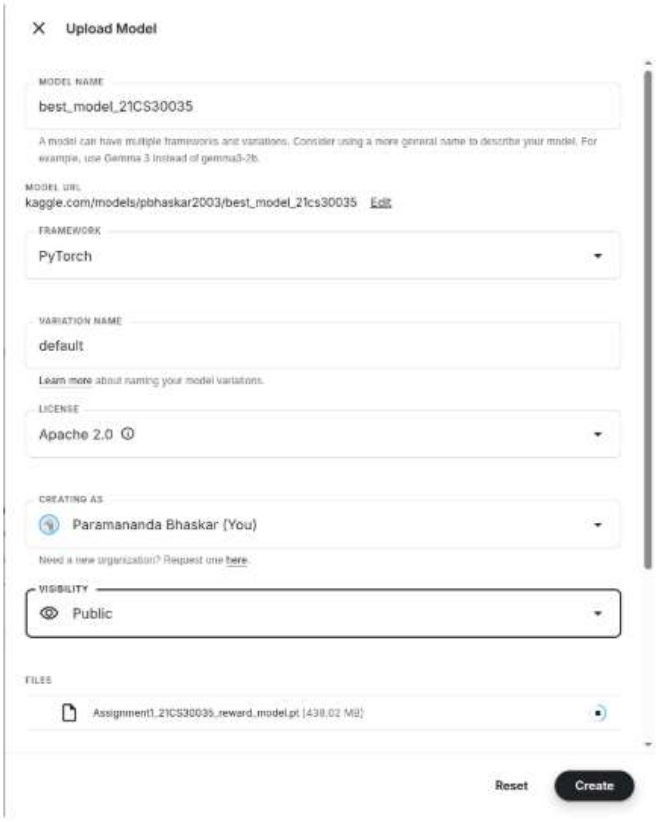
Submit a brief report containing:

1. **Results:**
   - **Baseline Comparison:** Present a clear table showing the **Test Top-5 Accuracy**, **Perplexity (PPL)**, **parameter counts** and *(optional)* training time efficiency for your RNN and LSTM models. Identify which model performed the best.
   - **Residual Connection Results:** Show the Test Top-5 Accuracy and PPL for your best baseline model and the new version with the residual (skip) connection. **State the percentage improvement.**
2. **Model Access:** You must save and share your final, best-performing model.
   - **File Format:** You are required to save the **entire model object** directly using the .pt file extension.

- **Example Code:** torch.save(model, 'best_model_<Roll_Number>.pt')
- ⚠️ **Submitting only the `state_dict` instead of the full model object will incur a penalty of 5% marks.**

- **Upload In Kaggle:** Upload your saved model file (best_model_<Roll_Number>.pt) to a public **Kaggle Model** as shown in the following images. (login to kaggle -> Create Model -> upload the .pt file with the configs as shown in the images).





3. Provide the full, **public** Kaggle URL where you have uploaded the best-performing model (e.g., https://www.kaggle.com/models/yourusername/your-model).
4. When you upload your model, make sure its visibility is set to **Public**. Submissions with a private model will not be accessible for evaluation and will incur a **penalty of 5% marks**.
5. **Only a single .pt file is allowed at the model URL** - uploading .zip, .h5 or other formats or multiple files will incur a **penalty of 5% marks.**

Submission Guidelines

- You have to use IPython Notebooks for coding. (Use Google Colab, Kaggle for running your assignments).
- Include small documentation on each cell of the notebook. Further, properly add comments to your code as and when required.
- **Important –** Make sure your IPython Notebook has the outputs from each cell. The plot(s) must be present in the submitted notebook as well. Absence of these results will lead to straight **deduction of 80% marks.**
- Manipulating metrics, misrepresenting results through methods that involve looking ahead in the dataset (e.g., using **bidirectional** RNN or LSTM) are **strictly prohibited**. Printing results and then altering code cell content to match is also forbidden. **Severe penalties** will be applied for such actions.
- Focus on the learning experience rather than excessively worrying about specific metric numbers.
- You have to submit the IPython notebook and Assignment report as mentioned above. Name each of them as NLP_Assignment_2_Roll_No.ipynb and NLP_Assignment_2_Roll_No.pdf, respectively.

  [**For eg:** NLP_Assignment_2_21CS30035.ipynb, NLP_Assignment_2_21CS30035.pdf]

- Submit these two files in **.zip format** on **Google Form**. **Note**: Your zip file must be named NLP_Assignment_2_Roll_No.zip and if you submit more than once, only your **latest submission** will be considered.

  [**For eg:** NLP_Assignment_2_21CS30035.zip]